

## **Module-1**

### **Session 5**

### **(TASKS)**

- 1. Write the Git command for each: initialize a repo, stage all files, commit with a message, push to remote, pull latest changes.**

**Ans:**

- Git is a distributed version control system that helps developers track changes in source code and collaborate on projects. Some commonly used Git commands are given below:

**a) Initialize a new Git repository: -**

This command creates a new Git repository in the current directory.

**Command: -**

```
git init
```

**b) Stage all files: -**

This command adds all modified and new files to the staging area before committing them.

**Command: -**

```
git add .
```

**c) Commit with a message: -**

This command saves the staged changes to the local repository with a descriptive message.

**Command: -**

```
git commit -m "Initial commit"
```

**d) Push changes to the remote repository: -**

This command uploads the committed changes from the local repository to the remote GitHub repository.

**Command: -**

```
git push origin main
```

**e) Pull the latest changes: -**

This command downloads and merges the latest changes from the remote repository into the local repository.

**Command: -**

```
git pull origin main
```

## Session 5 (TASKS)

### 2. What is the difference between git fetch and git pull? Answer in 2 lines.

**Ans:**

- Both **git fetch** and **git pull** are used to retrieve updates from a remote repository, but they work differently and serve different purposes.
- The **git fetch** command downloads the latest commits, branches, and updates from the remote repository without modifying the local working branch. It allows developers to review incoming changes before integrating them into their code. This is useful when developers want more control over the update process.
- On the other hand, the **git pull** command not only downloads the latest changes from the remote repository but also automatically merges those changes into the current local branch. This makes the local repository immediately up to date with the remote repository.

**Example Commands: -**

```
git fetch origin
```

```
git pull origin main
```

- In summary, **git fetch** only downloads updates, while **git pull** downloads and merges updates. Developers often use git fetch when they want to inspect changes before applying them and use git pull when they want to quickly synchronize their local repository with the remote repository.

### 3. Create a new GitHub repository, push any small text file from your local system, and share the repo link.

**Ans:**

- This task is a practical activity that must be performed using your own GitHub account and computer. Since I do not have access to your GitHub account or local system, I cannot create the repository on your behalf. However, the process is explained below.

**Step 1: Create a New Repository: -**

Log in to GitHub and click on **New Repository**. Enter a repository name such as **Git-Practice** and create the repository.

**Step 2: Create a Text File: -**

Create a file named **sample.txt** and add some text to it.

**Example content: -**

This is my first GitHub repository.

**Step 3: Initialize Git and Push the File: -**

## Session 5 (TASKS)

```
git init
git add .
git commit -m "Added sample text file"
git branch -M main
git remote add origin https://github.com/yourusername/Git-Practice.git
git push -u origin main
```

### Step 4: Share the Repository Link: -

After successfully pushing the file, GitHub will host the repository online. The repository link will look similar to: -

<https://github.com/yourusername/Git-Practice>

**Replace yourusername with your actual GitHub username. For your assignment, you should submit the actual repository URL created by you.**

### 4. What is the difference between a Fork and a Pull Request? Answer in 2 lines.

**Ans:**

- A **Fork** and a **Pull Request** are important concepts used in collaborative software development on GitHub, but they serve different purposes.
- A **Fork** is a personal copy of another user's repository that is created under your own GitHub account. Forking allows developers to experiment with changes, add new features, or fix bugs without affecting the original repository. Since the fork is independent, the developer has full control over it.
- A **Pull Request (PR)** is a request submitted to the owner of the original repository asking them to review and merge the changes made in a fork or another branch. It acts as a communication tool between developers and enables code review before integration.

**Example: -**

Suppose a developer forks an open-source project, fixes a bug, and commits the changes. The developer then creates a Pull Request so that the project maintainers can review the modifications and merge them into the original project.

**Key Difference: -**

- **Fork:** Creates a separate copy of a repository for independent development.
- **Pull Request:** Requests that changes be reviewed and merged into another repository or branch.

**Together, Forks and Pull Requests make it easier for developers around the world to contribute to open-source projects in a controlled and organized manner.**

## Session 5 (TASKS)

### 5. Create a new branch named feature-test, make one commit on it, and merge it back to the main branch. Write the commands you used.

**Ans:**

- Git branches allow developers to work on new features or bug fixes without affecting the main codebase. Once the work is completed and tested, the branch can be merged back into the main branch.

#### **Step 1: Create and Switch to the New Branch: -**

```
git checkout -b feature-test
```

This command creates a new branch named **feature-test** and immediately switches to it.

#### **Step 2: Make Changes to a File: -**

For example, add a new line to a file named sample.txt.

Added feature-test branch changes.

#### **Step 3: Stage the Changes: -**

```
git add .
```

This command adds all modified files to the staging area.

#### **Step 4: Create a Commit: -**

```
git commit -m "Added changes in feature-test branch"
```

This command saves the changes permanently in the feature-test branch with a descriptive commit message.

#### **Step 5: Switch Back to the Main Branch: -**

```
git checkout main
```

This command returns the user to the main branch.

#### **Step 6: Merge the Branch: -**

```
git merge feature-test
```

This command merges all commits from the feature-test branch into the main branch.

#### **Step 7: Delete the Branch (Optional): -**

```
git branch -d feature-test
```

This command removes the feature-test branch after its changes have been successfully merged.